

EC200U&EG91xU Series QuecOpen(SDK) **Wi-Fi Scan Development Guide**

LTE Standard Module Series

Version: 1.1

Date: 2023-09-05

Status: Released



At Quectel, our aim is to provide timely and comprehensive services to our customers. If you require any assistance, please contact our headquarters:

Quectel Wireless Solutions Co., Ltd.

Building 5, Shanghai Business Park Phase III (Area B), No.1016 Tianlin Road, Minhang District, Shanghai 200233, China

Tel: +86 21 5108 6236

Email: info@quectel.com

Or our local offices. For more information, please visit:

<http://www.quectel.com/support/sales.htm>.

For technical support, or to report documentation errors, please visit:

<http://www.quectel.com/support/technical.htm>.

Or email us at: support@quectel.com.

Legal Notices

We offer information as a service to you. The provided information is based on your requirements and we make every effort to ensure its quality. You agree that you are responsible for using independent analysis and evaluation in designing intended products, and we provide reference designs for illustrative purposes only. Before using any hardware, software or service guided by this document, please read this notice carefully. Even though we employ commercially reasonable efforts to provide the best possible experience, you hereby acknowledge and agree that this document and related services hereunder are provided to you on an “as available” basis. We may revise or restate this document from time to time at our sole discretion without any prior notice to you.

Use and Disclosure Restrictions

License Agreements

Documents and information provided by us shall be kept confidential, unless specific permission is granted. They shall not be accessed or used for any purpose except as expressly provided herein.

Copyright

Our and third-party products hereunder may contain copyrighted material. Such copyrighted material shall not be copied, reproduced, distributed, merged, published, translated, or modified without prior written consent. We and the third party have exclusive rights over copyrighted material. No license shall be granted or conveyed under any patents, copyrights, trademarks, or service mark rights. To avoid ambiguities, purchasing in any form cannot be deemed as granting a license other than the normal non-exclusive, royalty-free license to use the material. We reserve the right to take legal action for noncompliance with abovementioned requirements, unauthorized use, or other illegal or malicious use of the material.

Trademarks

Except as otherwise set forth herein, nothing in this document shall be construed as conferring any rights to use any trademark, trade name or name, abbreviation, or counterfeit product thereof owned by Quectel or any third party in advertising, publicity, or other aspects.

Third-Party Rights

This document may refer to hardware, software and/or documentation owned by one or more third parties ("third-party materials"). Use of such third-party materials shall be governed by all restrictions and obligations applicable thereto.

We make no warranty or representation, either express or implied, regarding the third-party materials, including but not limited to any implied or statutory, warranties of merchantability or fitness for a particular purpose, quiet enjoyment, system integration, information accuracy, and non-infringement of any third-party intellectual property rights with regard to the licensed technology or use thereof. Nothing herein constitutes a representation or warranty by us to either develop, enhance, modify, distribute, market, sell, offer for sale, or otherwise maintain production of any our products or any other hardware, software, device, tool, information, or product. We moreover disclaim any and all warranties arising from the course of dealing or usage of trade.

Privacy Policy

To implement module functionality, certain device data are uploaded to Quectel's or third-party's servers, including carriers, chipset suppliers or customer-designated servers. Quectel, strictly abiding by the relevant laws and regulations, shall retain, use, disclose or otherwise process relevant data for the purpose of performing the service only or as permitted by applicable laws. Before data interaction with third parties, please be informed of their privacy and data security policy.

Disclaimer

- a) We acknowledge no liability for any injury or damage arising from the reliance upon the information.
- b) We shall bear no liability resulting from any inaccuracies or omissions, or from the use of the information contained herein.
- c) While we have made every effort to ensure that the functions and features under development are free from errors, it is possible that they could contain errors, inaccuracies, and omissions. Unless otherwise provided by valid agreement, we make no warranties of any kind, either implied or express, and exclude all liability for any loss or damage suffered in connection with the use of features and functions under development, to the maximum extent permitted by law, regardless of whether such loss or damage may have been foreseeable.
- d) We are not responsible for the accessibility, safety, accuracy, availability, legality, or completeness of information, advertising, commercial offers, products, services, and materials on third-party websites and third-party resources.

Copyright © Quectel Wireless Solutions Co., Ltd. 2023. All rights reserved.

About the Document

Revision History

Version	Date	Author	Description
-	2021-03-04	Jensen FANG	Creation of the document
1.0	2021-10-21	Jensen FANG/ Terry TAO	First official release
1.1	2023-09-05	Sum LI	1. Updated the document name based on the unified QuecOpen naming. 2. Added the applicable modules EG912U-GL and EG915U series. 3. Added the Wi-Fi Scan error code enumeration member <code>QL_WIFISCAN_NO_SET_CB_ERR</code> (Chapter 2.3.1.1).

Contents

About the Document.....	3
Contents	4
Table Index.....	5
1 Introduction	6
2 Wi-Fi Scan APIs.....	7
2.1. Header File	7
2.2. API Overview	7
2.3. API Description.....	8
2.3.1. ql_wifiscan_open.....	8
2.3.1.1. ql_errcode_wifi_scan_e.....	8
2.3.2. ql_wifiscan_close	9
2.3.3. ql_wifiscan_option_set.....	9
2.3.3.1. ql_wifiscan_channel_e	10
2.3.4. ql_wifiscan_do.....	11
2.3.4.1. ql_wifi_ap_info_s	12
2.3.4.2. ql_wifiscan_result_s	12
2.3.5. ql_wifiscan_async	13
2.3.6. ql_wifiscan_register_cb.....	13
2.3.6.1. ql_wifiscan_callback	14
2.3.6.2. ql_wifiscan_ind_msg_s.....	14
3 Development Example of Wi-Fi Scan	15
3.1. Example Introduction.....	15
3.2. Auto-Start of Wi-Fi Scan Example	16
4 Appendix References	17

Table Index

Table 1: API Overview	7
Table 2: Related Document	17
Table 3: Terms and Abbreviations	17

1 Introduction

Quectel EC200U series and EG91xU family (EG912U-GL and EG915U series) modules support QuecOpen® solution. QuecOpen® is an embedded development platform based on RTOS system, which is intended to simplify the design and development of IoT applications. For more information on QuecOpen®, see **document [1]**.

This document is applicable to QuecOpen® solution based on SDK build environment, and introduces Wi-Fi Scan APIs and development example on Quectel EC200U and EG91xU family modules in QuecOpen® solution.

2 Wi-Fi Scan APIs

2.1. Header File

ql_wifi_scan.h, the header file of Wi-Fi Scan API, is in the *components\ql-kernel\inc* directory in SDK. Unless otherwise specified, all the header files mentioned in this document are in this directory.

2.2. API Overview

Table 1: API Overview

Functions	Description
<i>ql_wifiscan_open()</i>	Enables Wi-Fi Scan.
<i>ql_wifiscan_close()</i>	Disables Wi-Fi Scan.
<i>ql_wifiscan_option_set()</i>	Configures the scan parameters of Wi-Fi Scan.
<i>ql_wifiscan_do()</i>	Starts scanning in Wi-Fi Scan synchronization mode.
<i>ql_wifiscan_register_cb()</i>	Registers the callback function.
<i>ql_wifiscan_async()</i>	Starts scanning in Wi-Fi Scan asynchronization mode.

NOTE

Unless otherwise specified, all the Wi-Fi Scan APIs described in this document are not supported to call concurrently and the functions cannot be called in any callback function. Otherwise, the processing of subsequent messages will be affected.

2.3. API Description

2.3.1. ql_wifiscan_open

This function enables Wi-Fi Scan.

- **Prototype**

```
ql_errcode_wifi_scan_e ql_wifiscan_open (void);
```

- **Parameter**

None

- **Return Value**

See the error code in **Chapter 2.3.1.1** for details.

2.3.1.1. ql_errcode_wifi_scan_e

The error code enumeration of Wi-Fi Scan:

```
typedef enum
{
    QL_WIFISCAN_SUCCESS                = 0,
    QL_WIFISCAN_EXECUTE_ERR            = 1 | QL_DEV_ERRCODE_BASE,
    QL_WIFISCAN_MEM_ADDR_NULL_ERR,
    QL_WIFISCAN_INVALID_PARAM_ERR,
    QL_WIFISCAN_SEMAPHORE_WAIT_ERR,
    QL_WIFISCAN_MUTEX_TIMEOUT_ERR,
    QL_WIFISCAN_OPEN_FAIL,
    QL_WIFISCAN_BUSY_ERR,
    QL_WIFISCAN_ALREADY_OPEN_ERR,
    QL_WIFISCAN_NOT_OPEN_ERR,
    QL_WIFISCAN_HW_OCCUPIED_ERR,
    QL_WIFISCAN_NO_SET_CB_ERR,
}ql_errcode_wifi_scan_e;
```

- Member

Member	Description
QL_WIFISCAN_SUCCESS	Successful execution
QL_WIFISCAN_EXECUTE_ERR	Failed execution
QL_WIFISCAN_MEM_ADDR_NULL_ERR	Memory request failed.
QL_WIFISCAN_INVALID_PARAM_ERR	Invalid parameter(s).
QL_WIFISCAN_SEMAPHORE_WAIT_ERR	Semaphore waiting error.
QL_WIFISCAN_MUTEX_TIMEOUT_ERR	Error occurs at getting mutex.
QL_WIFISCAN_OPEN_FAIL	Error occurs at enabling Wi-Fi Scan.
QL_WIFISCAN_ALREADY_OPEN_ERR	Error occurs when enabling Wi-Fi Scan repeatedly.
QL_WIFISCAN_NOT_OPEN_ERR	Wi-Fi Scan is not enabled.
QL_WIFISCAN_BUSY_ERR	Wi-Fi Scan is busy, such as for scanning.
QL_WIFISCAN_HW_OCCUPIED_ERR	Hardware is occupied.
QL_WIFISCAN_NO_SET_CB_ERR	Callback function is not set.

2.3.2. ql_wifiscan_close

This function disables Wi-Fi Scan.

- Prototype

```
ql_errcode_wifi_scan_e ql_wifiscan_close (void);
```

- Parameter

None

- Return Value

See the error code in **Chapter 2.3.1.1** for details.

2.3.3. ql_wifiscan_option_set

This function configures the scan parameters of Wi-Fi Scan.

● Prototype

```
ql_errcode_wifi_scan_e ql_wifiscan_option_set (uint16_t expect_ap_cnt, uint8_t round, uint16_t scan_channel, uint32_t channel_time);
```

● Parameter

expect_ap_cnt:

[In] Maximum number of APs that can be scanned. Range: 1–300.

round:

[In] Number of scan cycles.

scan_channel:

[In] The scanned channel. Each bit from bit0 to bit12 represents a channel (bit0 represents channel 1 and bit12 represents channel 13). If one of the bits is set to 1, it indicates that this channel needs to be scanned. If bit0 to bit12 are all set to 0, it means all channels from 1 to 13 need to be scanned. See the channel information in **Chapter 2.3.3.1** for details.

channel_time:

[In] The maximum scan time per cycle per channel.

● Return Value

See the error code in **Chapter 2.3.1.1** for details.

2.3.3.1. ql_wifiscan_channel_e

The enumeration types related to the channels of Wi-Fi Scan:

```
typedef enum
{
    QL_WIFISCAN_CHANNEL_ALL_BIT    = 0x1FFF,
    QL_WIFISCAN_CHANNEL_ALL       = 0,
    QL_WIFISCAN_CHANNEL_ONE       = 0x0001,
    QL_WIFISCAN_CHANNEL_TWO       = 0x0002,
    QL_WIFISCAN_CHANNEL_THREE     = 0x0004,
    QL_WIFISCAN_CHANNEL_FOUR      = 0x0008,
    QL_WIFISCAN_CHANNEL_FIVE      = 0x0010,
    QL_WIFISCAN_CHANNEL_SIX       = 0x0020,
    QL_WIFISCAN_CHANNEL_SEVEN     = 0x0040,
    QL_WIFISCAN_CHANNEL_EIGHT     = 0x0080,
    QL_WIFISCAN_CHANNEL_NINE      = 0x0100,
    QL_WIFISCAN_CHANNEL_TEN       = 0x0200,
```

```

QL_WIFISCAN_CHANNEL_ELEVEN    = 0x0400,
QL_WIFISCAN_CHANNEL_TWELVE    = 0x0800,
QL_WIFISCAN_CHANNEL_THIRTEEN  = 0x1000,
QL_WIFISCAN_CHANNEL_TOTAL      = 13,      QL_WIFISCAN_CHANNEL_TOTAL
= QL_WIFISCAN_CHANNEL_THIRTEEN,
QL_WIFISCAN_CHANNEL_MAX,
}ql_wifiscan_channel_e;

```

● Member

Member	Description
<i>QL_WIFISCAN_CHANNEL_ALL_BIT</i>	Configure all channels based on bit
<i>QL_WIFISCAN_CHANNEL_ALL</i>	All channels
<i>QL_WIFISCAN_CHANNEL_ONE</i>	Channel 1
<i>QL_WIFISCAN_CHANNEL_TWO</i>	Channel 2
<i>QL_WIFISCAN_CHANNEL_THREE</i>	Channel 3
<i>QL_WIFISCAN_CHANNEL_FOUR</i>	Channel 4
<i>QL_WIFISCAN_CHANNEL_FIVE</i>	Channel 5
<i>QL_WIFISCAN_CHANNEL_SIX</i>	Channel 6
<i>QL_WIFISCAN_CHANNEL_SEVEN</i>	Channel 7
<i>QL_WIFISCAN_CHANNEL_EIGHT</i>	Channel 8
<i>QL_WIFISCAN_CHANNEL_NINE</i>	Channel 9
<i>QL_WIFISCAN_CHANNEL_TEN</i>	Channel 10
<i>QL_WIFISCAN_CHANNEL_ELEVEN</i>	Channel 11
<i>QL_WIFISCAN_CHANNEL_TWELVE</i>	Channel 12
<i>QL_WIFISCAN_CHANNEL_THIRTEEN</i>	Channel 13
<i>QL_WIFISCAN_CHANNEL_TOTAL</i>	Total number of channels
<i>QL_WIFISCAN_CHANNEL_MAX</i>	Invalid channel

2.3.4. ql_wifiscan_do

This function starts scanning in Wi-Fi Scan synchronization mode.

● Prototype

```
ql_errcode_wifi_scan_e ql_wifiscan_do (uint16_t *p_ap_cnt, ql_wifi_ap_info_s *p_ap_infos);
```

● Parameter

p_ap_cnt:

[Out] Number of scanned APs

p_ap_infos:

[Out] Information of all scanned APs. See **Chapter 2.3.4.1** for details.

● Return Value

See the error code in **Chapter 2.3.1.1** for details.

2.3.4.1. ql_wifi_ap_info_s

The structure of the scanned Wi-Fi APs:

```
typedef struct
{
    uint32_t bssid_low;
    uint16_t bssid_high;
    uint8_t channel;
    int8_t rssival;
}ql_wifi_ap_info_s;
```

● Parameter

Type	Parameter	Description
uint32_t	<i>bssid_low</i>	Low-order 4 bytes of Wi-Fi AP MAC address.
uint16_t	<i>bssid_high</i>	High-order 2 bytes of Wi-Fi AP MAC address.
uint8_t	<i>channel</i>	All channels.
int8_t	<i>rssival</i>	Signal strength.

2.3.4.2. ql_wifiscan_result_s

The structure of storing the results of all Wi-Fi APs scanned each time:

```
typedef struct
{
    uint32_t ap_cnt;
    ql_wifi_ap_info_s *ap_infos;
}ql_wifiscan_result_s;
```

● Parameter

Type	Parameter	Description
uint32_t	<i>ap_cnt</i>	The number of the scanned Wi-Fi APs.
<i>ql_wifi_ap_info_s</i>	<i>ap_infos</i>	The memory address where the scanned Wi-Fi AP specific information is located.

2.3.5. ql_wifiscan_async

This function starts scanning in Wi-Fi Scan asynchronous mode.

● Prototype

```
ql_errcode_wifi_scan_e ql_wifiscan_async (void);
```

● Parameter

None

● Return Value

See the error code in **Chapter 2.3.1.1** for details.

2.3.6. ql_wifiscan_register_cb

This function registers the callback function.

● Prototype

```
ql_errcode_wifi_scan_e ql_wifiscan_register_cb (ql_wifiscan_callback wifiscan_cb);
```

● Parameter

wifiscan_cb:

[In] The pointer of callback function.

- **Return Value**

See the error code in **Chapter 2.3.1.1** for details.

2.3.6.1. ql_wifiscan_callback

The pointer of Wi-Fi Scan callback function:

```
typedef void (*ql_wifiscan_callback)(ql_wifiscan_ind_msg_s *msg_buf);
```

2.3.6.2. ql_wifiscan_ind_msg_s

The structure of the message that the kernel informs App through the callback function:

```
typedef struct
{
    uint32_t msg_id;
    uint32_t msg_err_code;
    void *msg_data;
}ql_wifiscan_ind_msg_s;
```

- **Parameter**

Type	Parameter	Description
uint32_t	<i>msg_id</i>	Message ID.
uint32_t	<i>msg_err_code</i>	The returned error code.
void	<i>msg_data</i>	Specific data that is notified.

3 Development Example of Wi-Fi Scan

3.1. Example Introduction

The example file *wifi_scan_demo.c* of Wi-Fi Scan, provided in the SDK codes of EC200U and EG91xU family modules, is in the `components\ql-application\wifi_scan` directory. The entry function is *ql_wifiscan_app_init()* as below.

```
void ql_wifiscan_app_init(void)
{
    QIOSStatus err = QL_OSI_SUCCESS;
    20
    21 ql_rtos_task_create(&ql_wifiscan_app_task, QL_WIFISACN_TASK_STACK_SIZE, QL_WIFISACN_TASK_Prio, "wifiscan-DEMO", ql_wifiscan_app_thread, NULL, QL_WIFISACN_TASK_EVENT_CNT);
    22 err = ql_rtos_task_create(&ql_wifiscan_app_task, QL_WIFISACN_TASK_STACK_SIZE, QL_WIFISACN_TASK_Prio, "wifiscan-DEMO", ql_wifiscan_app_thread, NULL, QL_WIFISACN_TASK_EVENT_CNT);
    23 if(err != QL_OSI_SUCCESS)
    24 {
    25     QL_WIFISACN_DEMO_LOG("demo-task-created-failed");
    26 }
}
```

In the Wi-Fi Scan example, first, scan the AP hotspot information in Wi-Fi Scan synchronization mode and print the information. Then scan the AP hotspot information in Wi-Fi Scan asynchronization mode and print the information. Finally, the module enters the cyclic scanning status, and the above steps are repeated when the scanning event arrives.

```
static void ql_wifiscan_app_thread(void *param)
{
    27 ql_rtos_task_sleep_ms(15000); // sleep some time, then to do demo
    28 ql_wifiscan_synchro_complete_flow();
    29 ql_rtos_task_sleep_ms(5000);
    30 ql_wifiscan_async_start(); // start a async scan, and it should trigger one event sent at callback
    31
    32 while(1)
    33 {
    34     ql_event_t event = {0};
    35     36 if(ql_event_try_wait(&event) != 0)
    37     {
    38         continue;
    39     }
    40
    41     42 if(event.id == QUEC_WIFISCAN_EVENT_ASYNC_IND)
    43     {
    44         ql_wifiscan_close(); // close the wifiscan device for the async scan
    45         ql_rtos_task_sleep_ms(15000);
    46
    47         // next cycle demo
    48         ql_wifiscan_synchro_complete_flow();
    49         ql_rtos_task_sleep_ms(5000);
    50         ql_wifiscan_async_start();
    51     }
    52 }
} // end ql_wifiscan_app_thread
```


3.2. Auto-Start of Wi-Fi Scan Example

The Wi-Fi Scan example is not enabled by default, and please uncomment the codes in *ql_init_demo_thread* to enable it for test as shown in the figure below.

```

:  #ifndef QL_APP_FEATURE_SPI_NOR_FLASH
:  ...//ql_spi_nor_flash_demo_init();
:  #endif
:
:  #ifndef QL_APP_FEATURE_SPI_NAND_FLASH
:  ...//ql_spi_nand_flash_demo_init();
:  #endif
:
:  #if defined(QL_APP_FEATURE_WIFISCAN)
:  ...//ql_wifiscan_app_init();
:  #endif
:
:  #ifndef QL_APP_FEATURE_HTTP_FOTA
:  >> //ql_fota_http_app_init();
:  #endif
:
:  #ifndef QL_APP_FEATURE_FTP_FOTA
:  ...//ql_fota_ftp_app_init();
:  #endif
:
:  #ifndef QL_APP_FEATURE_DECODER
    
```



4 Appendix References

Table 2: Related Document

Document Name
[1] Quectel_EC200U&EG91xU_Series_QuecOpen(SDK)_CSDK_Quick_Start_Guide

Table 3: Terms and Abbreviations

Abbreviation	Description
API	Application Programming Interface
AP	Access Point
App	Application
ID	Identifier
MAC	Media Access Control Address
RTOS	Real Time Operating System
SDK	Software Development Kit
Wi-Fi	Wireless Fidelity